



**Faculty of Engineering and Technology
Computer Science Department**

COMP438 - Distributed Computing

Project Report No. 2

Hadoop

Prepared by:

Students Name:

IDs:

Ayah Abdalmuti

1212074

Somaya DarMousa

1210323

Instructor: Dr. Mohammad Helal.

Date: 22. June. 2025

Abstract

This project demonstrates how Apache Hadoop can be used to replace a centralized directory server by managing client IP and User ID data using the MapReduce framework. Clients store their information in text files, which are processed through Hadoop to enable peer-to-peer communication. The implementation showcases the integration of Java networking with Hadoop's distributed processing, resulting in a scalable and decentralized chat system.

Table of Contents

1. Introduction	4
2. How Hadoop Works	4
3. Core Components of Hadoop	5
4. The Hadoop Ecosystem	6
5. Requirements	6
6. How to Install Hadoop	7
7. Running MapReduce Jobs	11
8. Map Reduce in Hadoop:	14
Problems Faced and Solutions	23
Conclusion	24

1. Introduction

Apache Hadoop is an open-source framework designed to store and process large volumes of data (Big Data) in a distributed manner across clusters of computers. Hadoop enables data processing and analysis using simple programming models and is capable of scaling from a single machine to thousands of interconnected nodes. Each node provides local storage and computing power, allowing Hadoop to efficiently manage datasets ranging from gigabytes to petabytes.

2. How Hadoop Works

Hadoop distributes datasets across clusters of low-cost, commodity hardware. Processing is performed in parallel across multiple machines, increasing speed and efficiency.

The typical workflow in a Hadoop system follows these steps:

1. Data input by the client.
2. HDFS (Hadoop Distributed File System) stores the data and handles metadata.
3. MapReduce processes and transforms the data.
4. YARN schedules tasks and manages computing resources across the cluster.

Hadoop is built with the assumption that hardware failures (e.g., machine or rack failure) are common. The system is therefore designed to automatically handle these failures without interrupting operations.

3. Core Components of Hadoop

Hadoop consists of four core modules that form its foundational architecture:

➤ **Hadoop Distributed File System (HDFS):**

HDFS is the primary storage system of Hadoop. It stores data across multiple nodes, allowing each node to process data locally. This reduces network latency and ensures high-throughput access to data. HDFS does not require predefined schemas, offering greater flexibility.

➤ **Yet Another Resource Negotiator (YARN):**

YARN is the resource management layer in Hadoop. It allocates CPU, memory, and other resources across applications. YARN is responsible for job scheduling, tracking, and resource optimization.

➤ **MapReduce Programming Model:**

MapReduce is a programming model for processing large-scale datasets. It splits data into smaller chunks, which are processed in parallel across the cluster. Once processing is complete, the system combines the results into a final, more manageable dataset.

➤ **Hadoop Common:**

Hadoop Common includes shared utilities, libraries, and tools used by other Hadoop modules. It ensures that all components of the framework work together efficiently.

4. The Hadoop Ecosystem

Beyond its core modules, Hadoop has a growing open-source ecosystem that supports a wide range of tools and applications for collecting, storing, processing, analyzing, and visualizing Big Data. Some popular ecosystem components include:

- Apache Pig: A high-level platform for data analysis using a language similar to SQL.
- Apache Hive: A data warehouse infrastructure built on top of Hadoop that provides data summarization and query capabilities using HiveQL (SQL-like syntax).
- Apache HBase: A distributed, scalable NoSQL database that runs on top of HDFS.
- Apache Spark: A powerful, fast-processing engine for large-scale data analytics that can work independently or alongside Hadoop.
- Presto: A distributed SQL query engine designed for fast analytics.
- Apache Zeppelin: A web-based notebook for interactive data analytics and visualization.

5. Requirements:

- Java 8
- Apache Hadoop 3.3.4
- Apache Maven 3.6.3 or later
- Windows/Linux environment with command-line access

6. How to Install Hadoop

➤ Required Tools

1. **Java JDK:** Necessary to run Hadoop as it's built using Java.
2. **7Zip or WinRAR:** Used to unzip the Hadoop binary package.
3. **CMD or PowerShell:** Needed to test environment variables and run Hadoop.

➤ Step-by-Step Installation and Configuration:

Step 1: Download and Extract Hadoop

1. **Download the Hadoop tar ball** from the official Apache Hadoop website: [Hadoop Releases](#).
2. **Extract the downloaded Hadoop** using the following script:

```
tar -xzf hadoop-3.3.4.tar.gz  
mv hadoop-3.3.4 /usr/local/Hadoop
```

Step 2: Clone Winutils Repository

1. **Clone or download** the winutils repository.
2. **Copy the contents** of hadoop-3.3.4/bin into the extracted location of the Hadoop binary package.

Step 3: Install Java JDK

1. **Install Java JDK** if it is not already installed. You may need to sign up and log in to download it.
2. **Download JDK 8u261** from an alternative resource if necessary.

Step 4: Configure Environment Variables

1. **Open the Start Menu**, type 'environment', and press enter to open System Properties.
2. **Click the Environment Variables button.**

JAVA_HOME environment variable

1. Find the installation directory of Java, e.g., C:\Program Files\Java\jdk1.8.0_261.
2. Create a new User variable:
 - o **Variable name:** JAVA_HOME
 - o **Value:** C:\Program Files\Java\jdk1.8.0_261

PATH environment variable

1. Click Path then Edit.
2. Click New and add:
 - o C:\Users\YourUser\Documents\cp-master\hadoop-3.3.4\bin
 - o C:\Program Files\Java\jdk1.8.0_261\bin

Hadoop environment

1. Open %HADOOP_HOME%\etc\hadoop\hadoop-env.cmd.
2. Change the JAVA_HOME line to:
set JAVA_HOME=C:\PROGRA~1\Java\jdk1.8.0_261
3. Reopen CMD or PowerShell and verify that the Hadoop command is available:
hadoop -version

Step 5: Configure Hadoop

1. **Locate configuration files** in %HADOOP_HOME%\etc\hadoop.

Configure core-site.xml

1. Edit core-site.xml and add:


```
<configuration>
<property>
<name>fs.default.name</name>
<value>hdfs://0.0.0.0:19000</value>
</property>
</configuration>
```

Configure HDFS

1. Create directories for the namenode and datanode:
 - o C:\Users\YourUser\Documents\cp-master\hadoop-3.3.4\data\dfs\namespace_logs
 - o C:\Users\YourUser\Documents\cp-master\hadoop-3.3.4\data\dfs\data

2. Edit hdfs-site.xml and add:

```
<configuration>
<property>
<name>dfs.replication</name>
<value>1</value>
</property>
<property>
<name>dfs.namenode.name.dir</name>
<value>file:///C:/Users/YourUser/Documents/cp-master/hadoop-3.3.4/data/dfs/namespace_logs</value>
</property>
<property>
<name>dfs.datanode.data.dir</name>
<value>file:///C:/Users/YourUser/Documents/cp-master/hadoop-3.3.4/data/dfs/data</value>
</property>
</configuration>
```

Configure MapReduce and YARN

1. Edit mapred-site.xml and add:

```
<configuration>
```

```

<property>
<name>mapreduce.framework.name</name>
<value>yarn</value>
</property>
<property>
<name>mapreduce.application.classpath</name>

<value>%HADOOP_HOME%/share/hadoop/mapreduce/*,%HADOOP_HOME%/share/hadoop/mapreduce/lib/*,%HADOOP_HOME%/share/hadoop/common/*,%HADOOP_HOME%/share/hadoop/common/lib/*,%HADOOP_HOME%/share/hadoop/yarn/*,%HADOOP_HOME%/share/hadoop/yarn/lib/*,%HADOOP_HOME%/share/hadoop/hdfs/*,%HADOOP_HOME%/share/hadoop/hdfs/lib/*</value>
</property>
</configuration>

```

2. Edit yarn-site.xml and add:

```

<configuration>
<property>
<name>yarn.resourcemanager.hostname</name>
<value>localhost</value>
</property>
<property>
<name>yarn.nodemanager.aux-services</name>
<value>mapreduce_shuffle</value>
</property>
<property>
<name>yarn.nodemanager.env-whitelist</name>

<value>JAVA_HOME,HADOOP_COMMON_HOME,HADOOP_HDFS_HOME,HADOOP_CONF_DIR,CLASSPATH_PREPEND_DISTCACHE,HADOOP_YARN_HOME,HADOOP_MAPRED_HOME</value>
</property>
</configuration>

```

Step 6: Initialize HDFS and Fix Errors

1. Run:
hdfs namenode -format
2. If errors occur, download a fixed JAR file and overwrite hadoop-hdfs-3.3.4.jar in %HADOOP_HOME%\share\hadoop\hdfs.

Step 7: Start HDFS Daemons

1. Run:
%HADOOP_HOME%\sbin\start-dfs.cmd

Step 8: Start YARN Daemons

1. Open a command line with elevated permissions.
2. Uninstall the yarn package manager if it conflicts.
3. Run:
%HADOOP_HOME%\sbin\start-yarn.cmd

Step 9: Access Useful Web Portals

- **HDFS Namenode UI:** <http://localhost:9870/dfshealth.html#taboverview>
- **HDFS Datanode UI:** <http://localhost:9864/datanode.html>
- **YARN Resource Manager UI:** <http://localhost:8088>

Step 10: Shutdown YARN and HDFS Daemons

1. Run:
%HADOOP_HOME%\sbin\stop-dfs.cmd
%HADOOP_HOME%\sbin\stop-yarn.cmd

7. Running MapReduce Jobs

Step 1: Configure Extra Environment Variables

1. **HADOOP_CP environment variable:**
 - o Open CLI and type `hadoop classpath`.
 - o Create a new User variable:
 - **Variable name:** HADOOP_CP
 - **Value:** (output from `hadoop classpath`)

2. HDFS_LOC environment variable:

- o Create a new User variable:
 - **Variable name:** HDFS_LOC
 - **Value:** hdfs://localhost:19000

Step 2: Compile the Project

1. Run:

```
mkdir dist/  
javac -cp %HADOOP_CP% <some_directory>/*.java -d dist/
```

Step 3: Produce a JAR File

1. Run:

```
cd dist  
jar -cvf <application_name>.jar <some_directory>/*.class
```

Step 4: Copy Inputs to HDFS

1. Ensure HDFS and YARN daemons are running.
2. Run:

```
hadoop fs -copyFromLocal <some_directory>/input  
%HDFS_LOC%/input  
hadoop fs -ls %HDFS_LOC%/input
```

Step 5: Run the MapReduce Job

1. Run:

```
hadoop jar <application_name>.jar <pkg_name>.<class_name>  
%HDFS_LOC%/input %HDFS_LOC%/output
```

Step 6: Copy Outputs to Local Machine

1. Run:

```
hadoop fs -copyToLocal %HDFS_LOC%/output  
<some_output_directory>
```

➤ How to operate it?

1.Starting and Stopping Hadoop Services

Starting Services:

HDFS (Hadoop Distributed File System): Code: **start-dfs.sh**

YARN (Yet Another Resource Negotiator)services: Code: **start-yarn.sh**

Stopping Services:

HDFS:Stop NameNode and DataNode services: Code :**stop-dfs.sh**

YARN:Stop ResourceManager and NodeManager services:Code:**stop-yarn.sh**

2. Monitoring Hadoop Cluster

Web Interfaces:

NameNode Web UI: Monitor HDFS status and browse the file system:

- o Access http://namenode_hostname:9870

ResourceManager Web UI: Monitor YARN applications and cluster resources:

- o Access http://resourcemanager_hostname:8088

Command Line:

Check running applications: Code: yarn application -list

Check node status: Code: yarn node -list

3.Managing Hadoop Data

HDFS Commands:

List files and directories: Code: hdfs dfs -ls /

Upload files to HDFS: Code: hdfs dfs -put /local/path /hdfs/path

Download files from HDFS: Code: hdfs dfs -get /hdfs/path /local/path

Remove files from HDFS: Code: hdfs dfs -rm /hdfs/path

4. Troubleshooting Hadoop

Log Files:

Hadoop logs: Located in \$HADOOP_HOME/logs/, check logs for NameNode, DataNode, ResourceManager, and NodeManager.

Common Issues:

Permission problems: Ensure correct permissions in HDFS and local filesystem.

Configuration errors: Verify and correct configuration files (coresite.xml, hdfs-site.xml, yarn-site.xml, etc.).

Resource allocation: Adjust memory and CPU settings (mapreduce.map.memory.mb, mapreduce.reduce.memory.mb, etc.) as needed.

5. Scaling and Managing the Cluster

Scaling:

- **Add nodes:** Expand cluster capacity by adding new nodes and configuring them to join the existing Hadoop setup.

Resource Management:

- **Optimize YARN:** Adjust configuration (yarn-site.xml, capacity-scheduler.xml, etc.) for efficient resource utilization and scheduling.

8. Map Reduce in Hadoop:

MapReduce is one of the core components of the Hadoop framework, alongside HDFS and YARN. While HDFS handles the distributed storage of large data files across clusters, MapReduce is responsible for processing that data efficiently. The MapReduce model consists of two main phases:

Map phase: This phase processes the input data and transforms it into intermediate key-value pairs.

Reduce phase: This phase takes the intermediate key-value pairs and aggregates or summarizes them to produce the final output.

This two-step process enables scalable, parallel processing of massive datasets across multiple nodes in a Hadoop cluster.

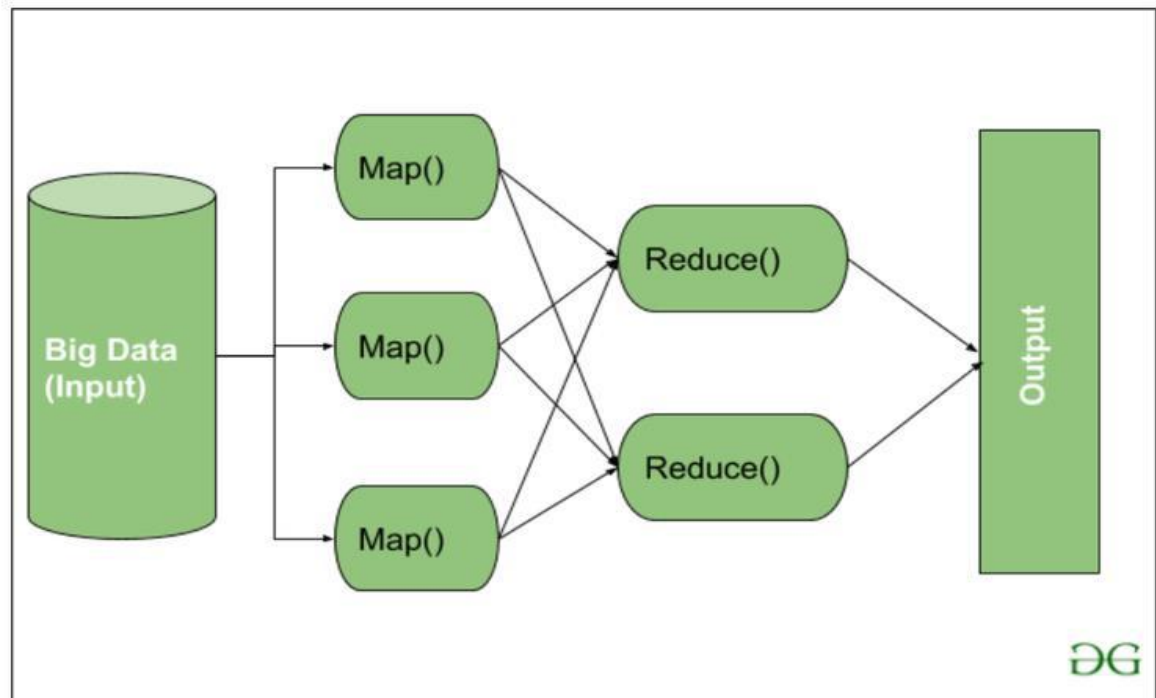


Figure 1: Map and Reduce interfaces

To understand how MapReduce operates within the Hadoop framework, consider a simple example. Suppose we have a file named `sample.txt` that contains the following 8 lines of text:

```
Hello my name is Ayah
What can I do for you
Do you need any help
Are you a developer
Do you enjoy coding
Are you preparing for interviews
What are your hobbies
What are your goals
```

Although Hadoop is designed to process large-scale data (gigabytes to petabytes), this small file helps illustrate the concept.

Step 1: File Storage with HDFS

When this file is stored in the Hadoop Distributed File System (HDFS), it is automatically split into fixed-size blocks called **input splits**. In this example, we assume it is split into 4 parts:

first.txt: lines 1–2

second.txt: lines 3–4

third.txt: lines 5–6

fourth.txt: lines 7–8

Each split is stored across **DataNodes**, while the **NameNode** keeps metadata (e.g., block location, file structure).

Step 2: Submitting a MapReduce Job

Suppose a user wants to process the content of sample.txt using a MapReduce job packaged in a JAR file named query.jar. The job is submitted using the following command:

```
hadoop jar query.jar DriverCode sample.txt result.output
```

query.jar: contains the compiled MapReduce logic.

sample.txt: input file.

result.output: directory to store the output.

Step 3: Execution Flow (Map Phase)

The job is received by the ResourceManager (previously known as JobTracker in Hadoop 1.x), which coordinates with NodeManagers (formerly TaskTrackers) to assign map tasks to nodes where data blocks reside.

Each of the four input splits is processed by one Mapper in parallel. The Map function transforms raw text into intermediate key-value pairs, for example: ("what", 1), ("can", 1), ("you", 1)

Step 4: Reduce Phase

After the map phase completes, the Reducer takes over. It aggregates values of similar keys from all mappers to produce the final result. For example:

("you", [1, 1, 1]) → ("you", 3)

This output is written to the specified output directory, typically as part-r-00000.

Fault Tolerance

Hadoop ensures reliability. If a node processing one input split fails, the ResourceManager reschedules the task on another node that holds a replica of the data. Nodes send heartbeats to report health and availability every few seconds. If a node is unresponsive beyond a timeout threshold, it is assumed dead, and recovery is initiated.

MapReduce efficiently processes distributed data by parallelizing tasks using mappers and aggregating results using reducers. The combination of HDFS for storage and MapReduce for processing enables Hadoop to handle massive-scale data reliably and efficiently.

➤ Explain the steps of implementing our project?

Step 1: Setup Hadoop Environment

- Install and configure Hadoop as described above.

Step 2: Implement the MapReduce Job

1. IPMapper Class: This Mapper reads each line of input containing a User ID and an IP address, splits them, and outputs the User ID as the key and the corresponding IP address as the value. This prepares the data for further processing in the Reduce phase.

```
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
import java.io.IOException;
```

```
public class IPMapper extends Mapper<Object, Text, Text, Text> {
```

```
private Text userID = new Text();
private Text ip = new Text();
```

@Override

protected void map(Object key, Text value, Context context) throws
IOException, InterruptedException {

 // Each line contains: UserID IP_Address

 String line = value.toString().trim();

 String[] parts = line.split("\\s+");

 if (parts.length == 2) {

 userID.set(parts[0]);

 ip.set(parts[1]);

 context.write(userID, ip);

 }

 }

}

2. IPReducer Class: This Reducer receives each User ID as the key and all associated IP addresses as values. It simply outputs each User ID paired with its corresponding IP addresses without modification, preparing the data for final storage or further processing.

```
import org.apache.hadoop.io.Text;
```

```
import org.apache.hadoop.mapreduce.Reducer;
```

```
import java.io.IOException;
```

```
public class IPReducer extends Reducer<Text, Text, Text, Text> {
```

@Override

protected void reduce(Text key, Iterable<Text> values, Context context)

throws IOException, InterruptedException {

 for (Text ip : values) {

 context.write(key, ip);

 }

 }

}

3. IPDriver Class: This class configures and runs the Hadoop MapReduce job. It sets the Mapper and Reducer classes (IPMapper and IPReducer), specifies the input and output data types, and manages the input/output file paths.

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class IPDriver {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Usage: IPDriver <input path> <output path>");
            System.exit(-1);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "IP Mapper");
        job.setJarByClass(IPDriver.class);

        job.setMapperClass(IPMapper.class);
        job.setReducerClass(IPReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

4. Client Class: This class represents a chat client that loads user IP addresses from the Hadoop MapReduce output file. It starts a server to listen for incoming chat messages and allows initiating chat connections to other clients using their IP addresses from the loaded map.

```
import java.io.*;
import java.net.*;
import java.util.*;

public class Client {
    private String name;
    private Map<String, String> ipMap = new HashMap<>(); // UserID -> IP

    public Client(String name) {
        this.name = name;
    }

    public void loadIPsFromFile(String filepath) throws IOException {
        BufferedReader br = new BufferedReader(new FileReader(filepath));
        String line;
        while ((line = br.readLine()) != null) {
            String[] parts = line.trim().split("\\s+");
            if (parts.length == 2) {
                ipMap.put(parts[0], parts[1]);
            }
        }
        br.close();
    }

    public void startServer(int port) {
        new Thread(() -> {
            try (ServerSocket serverSocket = new ServerSocket(port)) {
                System.out.println(name + " listening on port " + port);
                while (true) {
                    Socket clientSocket = serverSocket.accept();
                    new Thread(() -> handleClient(clientSocket)).start();
                }
            }
        }).start();
    }
}
```

```

        } catch (IOException e) {
            e.printStackTrace();
        }
    }).start();
}

private void handleClient(Socket socket) {
    try (BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()))) {
        String line;
        while ((line = in.readLine()) != null) {
            System.out.println("Message: " + line);
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}

public void chat(String targetName, int port) {
    String targetIP = ipMap.get(targetName);
    if (targetIP == null) {
        System.out.println("Target user not found.");
        return;
    }
    try (Socket socket = new Socket(targetIP, port);
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
        Scanner scanner = new Scanner(System.in)) {
        System.out.println("Chatting with " + targetName);
        while (true) {
            String msg = scanner.nextLine();
            out.println(name + ": " + msg);
        }
    } catch (IOException e) {
        System.out.println("Could not connect to " + targetName);
    }
}

```

```

public static void main(String[] args) throws IOException {
    if (args.length != 3) {
        System.out.println("Usage: java Client <YourName> <YourPort>
<PathToHadoopOutputFile>");
        return;
    }

    Client client = new Client(args[0]);
    client.loadIPsFromFile(args[2]);
    client.startServer(Integer.parseInt(args[1]));

    Scanner scanner = new Scanner(System.in);
    while (true) {
        System.out.println("Enter target username to chat or type 'exit' to quit:");
        String target = scanner.nextLine();
        if (target.equalsIgnoreCase("exit")) break;
        client.chat(target, Integer.parseInt(args[1]));
    }
}

```

Step 3: Compile and Package the Code

```

javac -classpath `hadoop classpath` -d IPProject *.java
jar -cvf IPProject.jar -C IPProject/ .

```

Step 4: Run the MapReduce Job

1. Upload input file to HDFS

```

hdfs dfs -mkdir /input
hdfs dfs -put localfile.txt /input

```

2. Run the MapReduce job

```

hadoop jar IPProject.jar projhadoop.IPDriver /input /output

```

3. Run the Client program to use output results

```

java -cp IPProject.jar projhadoop.Client /output

```

Problems Faced and Solutions

During the development and deployment of our project, we encountered several technical issues that required troubleshooting and problem-solving. One major challenge was configuring the system environment variables for Hadoop and Java. To resolve this, we manually updated the PATH variable to include the correct bin directories and verified the installation using `hadoop -version` and `java -version` commands.

Another issue was related to starting the YARN daemons, which sometimes failed due to misconfigurations. We addressed this by carefully reviewing `yarn-site.xml` and other configuration files, ensuring all prerequisites were met, and consulting Hadoop documentation and logs for detailed error analysis. These steps helped us ensure stable operation of the Hadoop environment and successful execution of the MapReduce job.

Conclusion

In conclusion, the project successfully demonstrated how Hadoop can be used to manage client data in a distributed environment. By replacing the centralized Directory Server with MapReduce-based processing of IP and User ID files, we were able to build a decentralized and scalable communication system. All components, including Mapper, Reducer, Driver, and client communication, worked as expected after proper setup. The project helped us gain hands-on experience with Hadoop and Java networking, and provided insights into real-world distributed systems design.